CSE 490/590: Project 1

Design and Implementation of a 16-bit Single-Cycle Processor

Justin Downer (`jrdowner`) Sajjad Ali (`sali22`) Vishal Kishore Kumar (`vishalki`)

April 4, 2026

1 Synopsis

This report documents the design, simulation, and hardware synthesis of a 16-bit, single-cycle, non-pipelined processor [1][cite: 1]. The architecture is byte-addressed with a 16-bit word size and supports 10 mandatory instructions across R-type, I-type, and J-type formats [1][cite: 2]. The processor was developed using Verilog HDL and verified using both Vivado simulations and the Xilinx Basys 3 FPGA board [2], [3][cite: 3].

2 Work Distribution

While specific modules were contributed by team members, the vast majority of the architectural design, system integration, and hardware verification was performed individually[cite: 4].

Table 1: Division of tasks among group members.

Member	**Role**	**Contribution Details**
Justin Downer	Lead Architect	Authored behavioral Verilog for the ALU, Control Unit, and Register File [1][cite: 5, 6].
Justin Downer	System Integration	Developed the structural top-level wrapper using Vivado Block Designer and managed all IP interconnections [3][cite: 7].
Justin Downer	Hardware Lead	Managed XDC mapping, bitstream generation, and 7-segment display integration for hardware demo [2][cite: 8].
Team Members	Contributors	Developed supplementary logic modules[cite: 9].

3 Implementation Details

3.1 Design Methodology

The processor integration utilized **Vivado's Block Designer** to visualize the datapath and component interconnections[cite: 10]. A structural Verilog file was maintained via the **Integrated HDL Wrapper** feature, ensuring that any changes to the block design were automatically reflected in the top-level code [3][cite: 11]. This reduced manual wiring errors while allowing individual components to be designed using **behavioral Verilog** [1][cite: 12].

3.2 Instruction Fetching and Memory Alignment

To maintain byte-addressing, the **Program Counter (PC)** increments by 2 for each sequential instruction [1][cite: 13]. An **Adder** module computes the next PC value before it is passed to Instruction Memory[cite: 14]. The Instruction Memory is initialized via the `readmemh` command, loading `vbit.mem` and slicing the 16-bit output into appropriate bit-fields for the Control Unit, Register File, and Sign Extension [1][cite: 15].

3.3 Branch and Jump Control Logic

The branching mechanism uses a specialized **XOR-based logic path** to handle both `BEQ` and `BNE` instructions [1][cite: 16]. The **Zero Flag** from the ALU is XORed with the `BNE` signal from the Control Unit[cite: 17]; the result is ANDed with the Branch Flag to drive the `PCSrc` multiplexer[cite: 18]. A custom **ShiftLeft1** module doubles the effective range of signed immediates while keeping the PC word-aligned [1][cite: 19].

3.4 Sign Extension and ALU Control

Two distinct **Sign Extension** modules handle the 4-bit signed immediate (I-type) and the 12-bit signed immediate (J-type) [1][cite: 20]. A dedicated **ALU Control** module decodes function bits for R-type instructions (e.g., `ADD`, `SUB`, `SLL`, `AND`), improving modularity and simplifying the primary Control Unit [1][cite: 21].

4 Simulation Results

The following waveforms demonstrate the functional correctness of the 10 mandatory instructions within the Vivado simulation environment[cite: 22, 23].

4.1 CPU Block Diagram

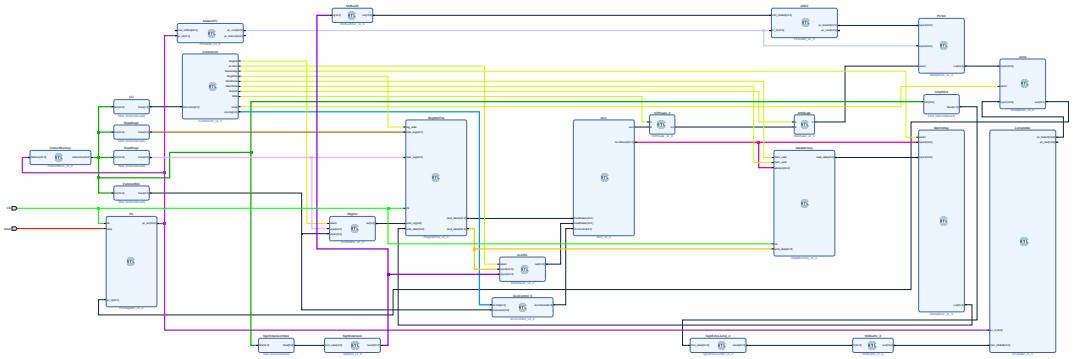


Figure 1: Top-level structural block diagram generated from Vivado IP Integrator.

4.2 Instruction Waveforms

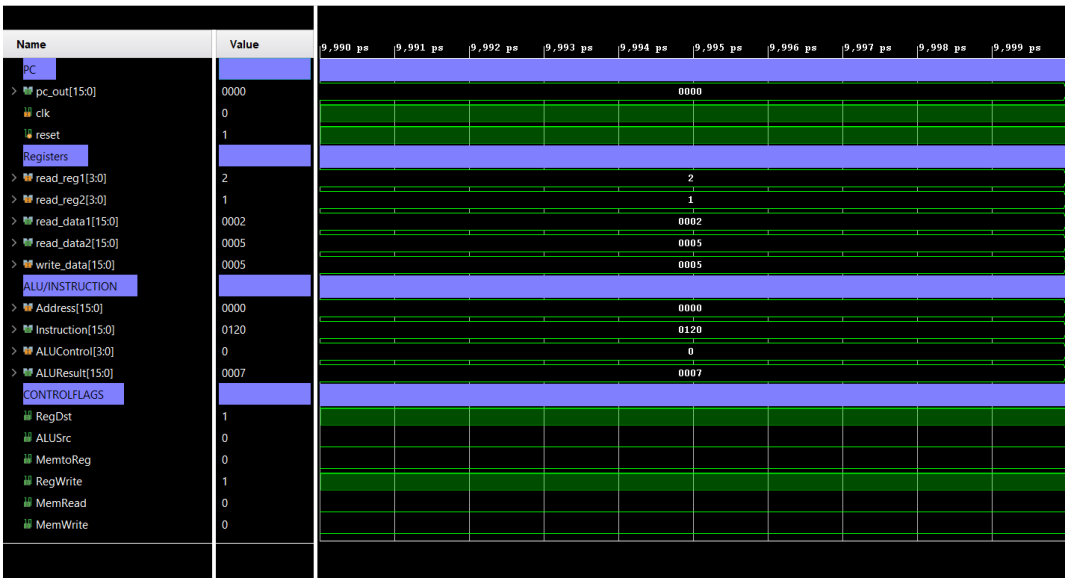


Figure 2: Waveform for R-type `ADD` instruction.

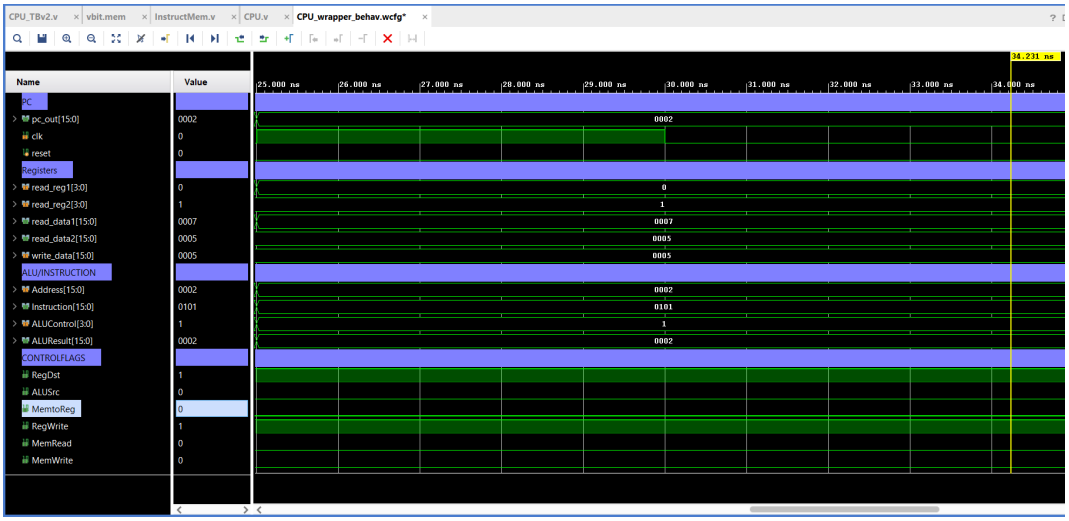


Figure 3: Waveform for R-type `SUB` instruction.

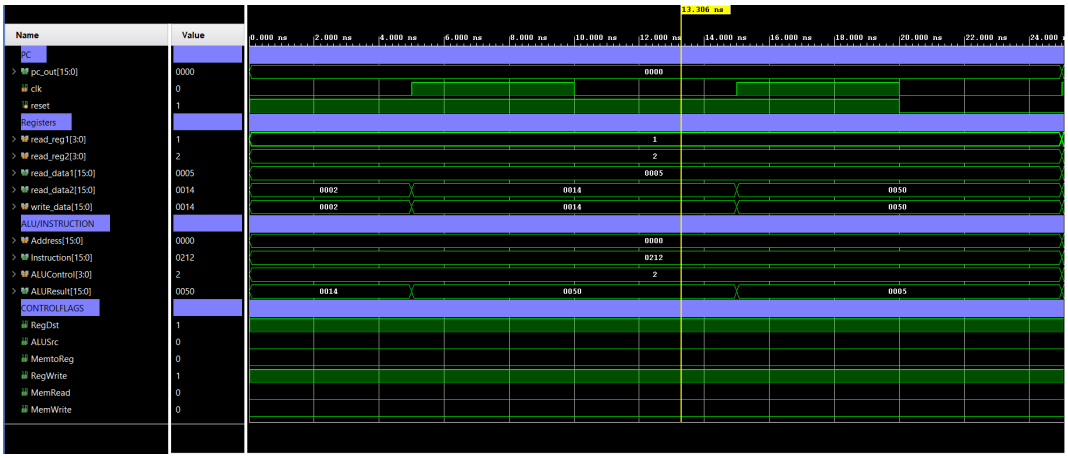


Figure 4: Waveform for R-type `SLL` instruction demonstrating shift-left logic.

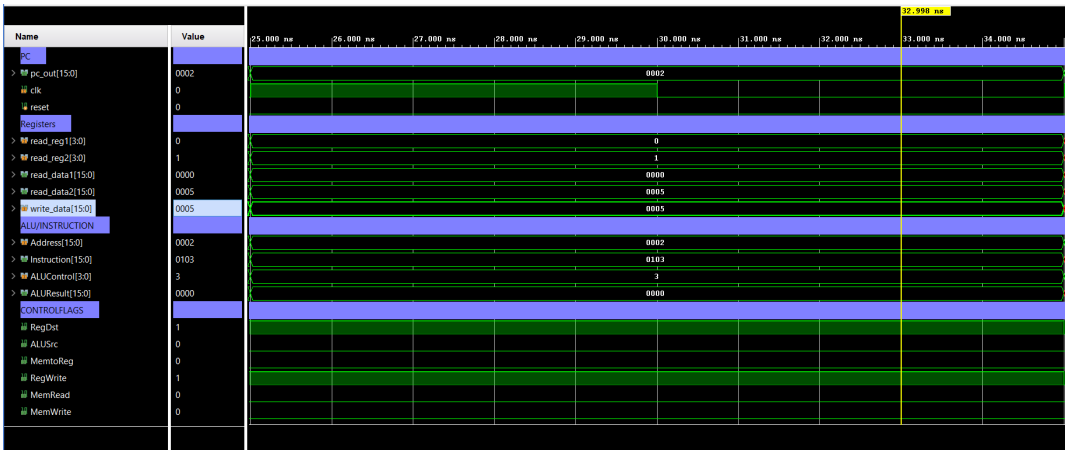
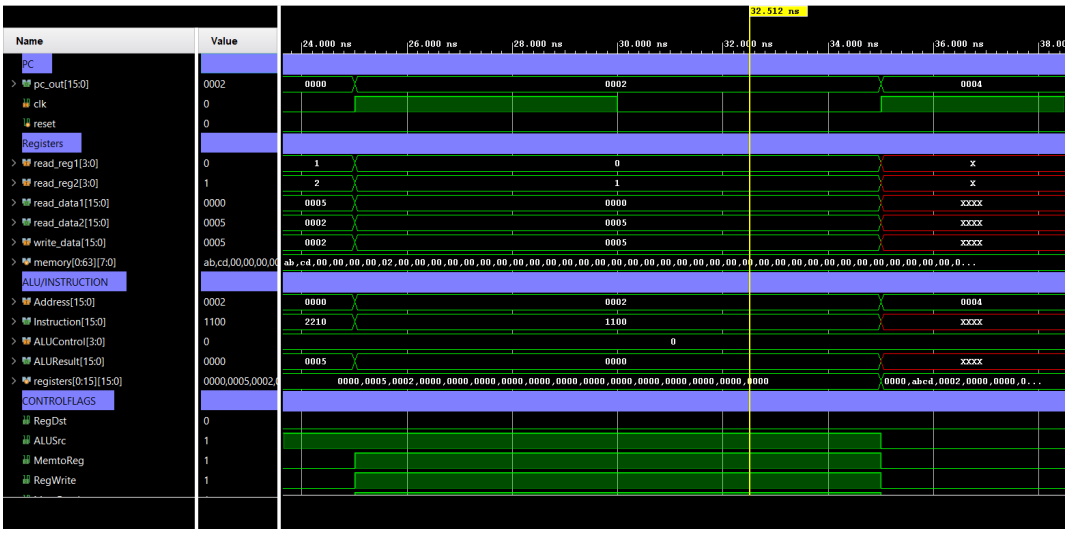


Figure 5: Waveform for R-type `AND` instruction.


32.512 ns
Name
Value
PC
pc_out[15:0] 0002
clk 0
reset 0
Registers
read_reg1[3:0] 0
read_reg2[3:0] 1
read_data1[15:0] 0000
read_data2[15:0] 0005
write_data[15:0] 0005
memory[0:63][7:0] ab,cd,00,00,00,0
ab,cd,00,00,00,00,02,00,00,00,00

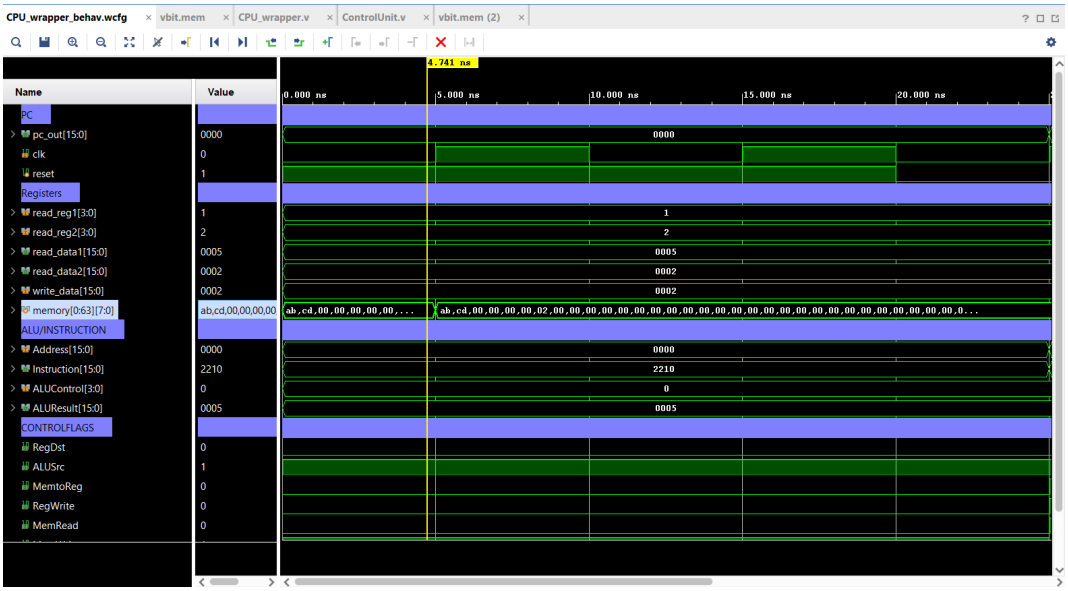


Figure 7: Waveform for I-type `SW` instruction showing register-to-memory data transfer.

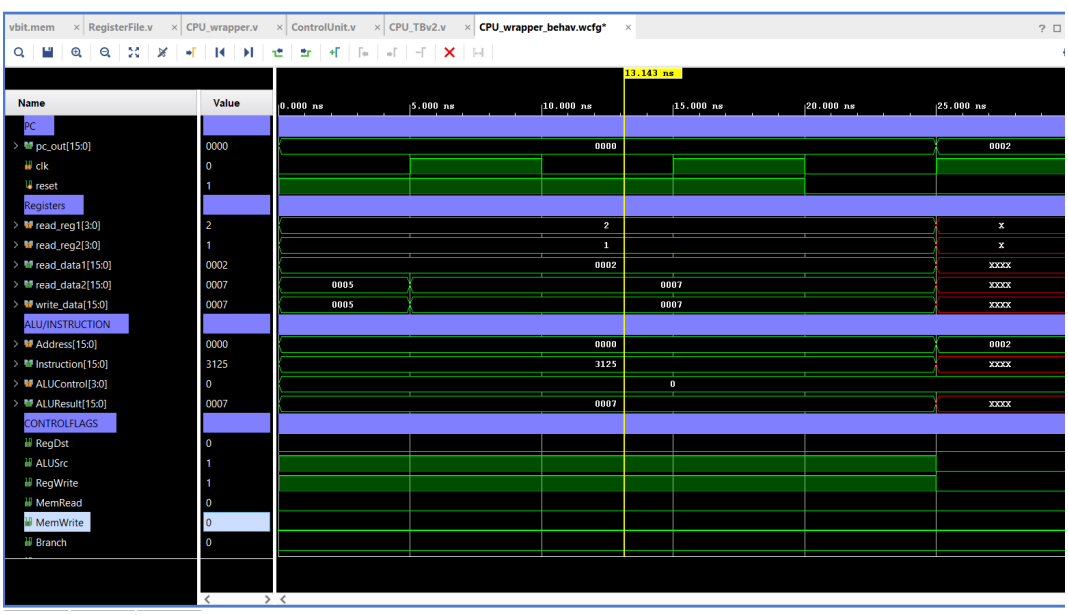


Figure 8: Waveform for I-type `ADDI` instruction using the 4-bit sign-extended immediate.

Figure 9: Waveform for I-type `BEQ` instruction verifying PC-relative branching.

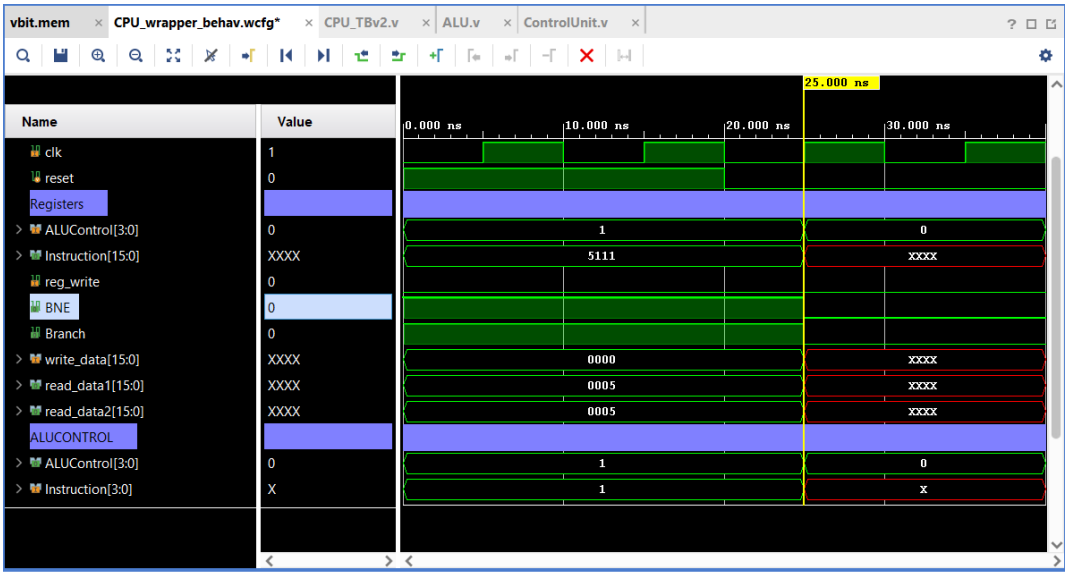


Figure 10: Waveform for I-type `BNE` instruction verifying the XOR-based branch logic.

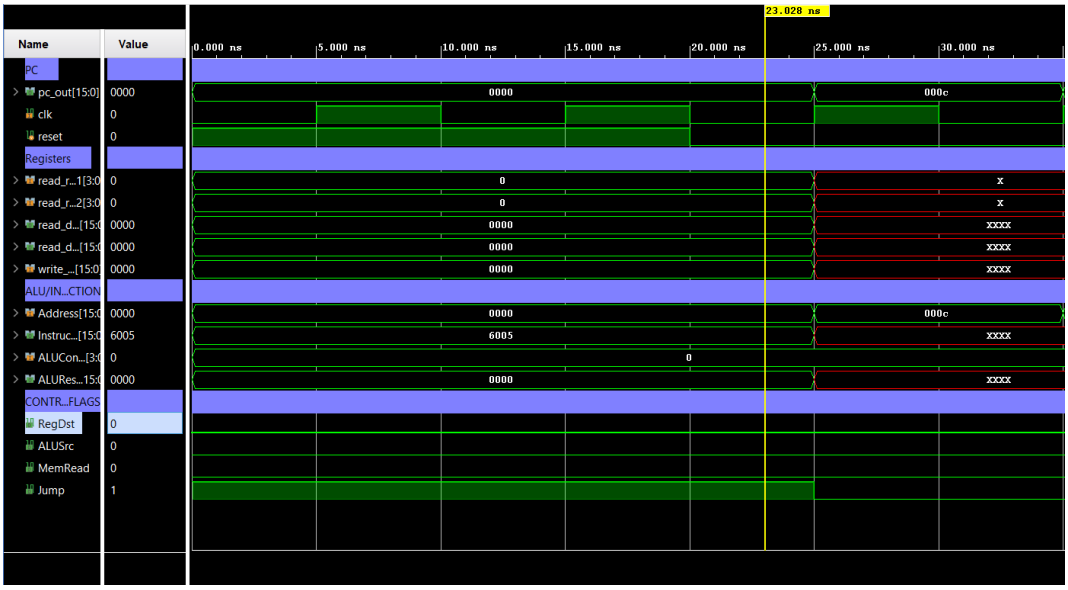


Figure 11: Waveform for J-type `JMP` instruction demonstrating PC-relative jump addressing.

5 Hardware Verification

5.1 Memory Initialization and Test Strategy

Data Memory was initialized using a Verilog `initial` block to hardcode test values for immediate verification of `LW` and `SW` instructions [1][cite: 22]. Success is defined as the architectural state matching expected values across all 10 mandatory instructions [1][cite: 23].

5.2 Hardware Visualization

To bridge the gap between internal logic and physical observation, the 16-bit **ALUResult** is mapped to the Basys 3 board's **7-segment display**, providing a hexadecimal representation of calculations during the live demonstration [2][cite: 24].

5.3 Synthesis and Implementation

The design passed through the Vivado **Synthesis** and **Implementation** flows without timing violations [3][cite: 25]. The absence of critical errors confirms that the logic is physically routable on the Artix-7 FPGA [3][cite: 26].

6 Conclusion

The successful implementation of this 16-bit processor demonstrates a complete design cycle — from ISA specification to FPGA synthesis[cite: 27]. The modular approach ensured stable execution in both simulation and hardware environments [1], [3][cite: 27].

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2020.

[2] Digilent, *Basys 3 FPGA board reference manual*, 2026. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[3] Xilinx, *Vivado design suite user guide*, 2026. [Online]. Available: https://docs.xilinx.com/r/en-US/ug892-vivado-design-flows-overview